Minigames (Short Guide)

Build a web game that teaches Python, with levels, quizzes and unlocking

Student tutorial: follow the steps in order and build the project as you read.

Contents

What You're Building **3**

Setup 3

The Big Picture 3

Build the Features **5**

Feature 1: Show a Lesson 5

Feature 2: Write the Loops Level 5

Feature 3: Check Practice Answers 5

Feature 4: Score the Quiz 6

Feature 5: Unlock the Next Level 6

Optional Extensions 6

What You're Building

Minigames is a small web game that teaches Python. A player works through five levels (Print, Variables, Conditionals, Loops, Functions). Every level has four tabs:

1. **Learn**: a short lesson with an example.
2. **Practice**: type one line of Python and the game checks it.
3. **Debug**: look at broken code and pick what is wrong.
4. **Quiz**: multiple-choice questions.

Finish all three activities in a level and the **next level unlocks**. A stuck player can press **Get a hint** for a nudge from an AI tutor.

The folder you were given (`minigames_smaller`) already launches, and four of the five levels are finished. You will write **five pieces** that make the game work and add a level of your own.

Setup

Open a terminal inside the `minigames_smaller` folder, then run these one at a time.

Mac:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

Windows (PowerShell or Command Prompt):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Open the address it prints (usually `http://127.0.0.1:7860`). To stop the game press `Ctrl + C` . **After you edit a Python file, stop and restart the game** to see your change. Each time you open a new terminal, run the activate line again.

> **TIP**
> On Windows, if `python` is not found try `py` . If PowerShell blocks the activate script, use Command Prompt instead.

AI hints are optional. The game works without a key (it shows built-in hints). To turn on real AI hints, get a key at aistudio.google.com/apikey, copy `.env.example` to `.env` , and set `GEMINI_API_KEY=` to your key. Never paste a key into a Python file or share your `.env` .

The Big Picture

When you click a button, information travels like this:

```
Browser (the Gradio page)
      |  you click a button
      v
   app.py   <-- the handler for that button runs
      |
      +--> levels.py       the content: lessons, exercises, bugs, quizzes
      +--> challenges.py   checks your answers
      +--> progress.py     remembers what you finished, unlocks levels
      +--> ai_helper.py    hints (AI if you have a key, built-in if not)
      |
      v
The result appears on the page
```

File	What it does
`app.py`	Builds the screen and runs a small *handler* function for each button. Finished.
`levels.py`	Holds every level as dictionaries and builds the lesson text. **You edit this.**
`challenges.py`	Decides if an answer is right, never running the student's code. **You edit this.**
`progress.py`	Keeps one dictionary of what is done and unlocked. **You edit this.**
`ai_helper.py`	The only file that talks to the AI. Finished.

Progress lives in memory, so **refreshing the page resets it**. Each handler returns the updated progress, and Gradio saves it.

> **TIP**
> `progress.py` has a switch, `UNLOCK_ALL_FOR_TESTING`. Set it to `True` while you build so every level is open. Set it back to `False` in Feature 5, or unlocking never gets used.

Build the Features

Feature 1: Show a Lesson

File: `levels.py`, function `get_lesson`

Goal: The Learn tab shows the title, explanation and example of whichever level is selected.

Your Task: Write `get_lesson(level_id)`. It receives an id such as `"print"` and returns one Markdown string: the title as a `##` heading, the explanation, then the example inside a ```` ```python ```` code block. If the id is not in `LEVELS`, return a friendly message instead of crashing. If a level's explanation still starts with `"TODO"`, return a "coming soon" message.

Hint: `LEVELS` is a dictionary of dictionaries, so `LEVELS["print"]["title"]` gets one value. Use an f-string to build the text, `in` to check a key exists, and `.startswith()` on a string.

Test: Set `UNLOCK_ALL_FOR_TESTING = True` in `progress.py`, restart, and pick each level. Levels 1, 2, 3 and 5 should show real lessons. Level 4 should say "coming soon".

Feature 2: Write the Loops Level

File: `levels.py`, the `"loops"` entry in `LEVELS`

Goal: Add your own lesson, practice question, bug and quiz so the Loops level is playable.

Your Task: Replace every `TODO` in `LEVELS["loops"]`. Copy the shape of the Conditionals level. You need:

- an `explanation` and an `example` about `for` and/or `while` loops
- an `exercise`: a `prompt` plus `accepted_answers`, a list of every correct way to write it (the game compares text, so the answer must fit on one line)
- a `debug` puzzle: buggy code with **one** clear bug, a question, four `choices`, and the `correct` one
- a `quiz` of 2 or 3 questions, each with `choices` and the right `answer`

Hint: In `debug`, `correct` and in each quiz question `answer` must be **spelled exactly like one of the choices**, or nobody can ever get it right. A missing comma between dictionary entries is the most common error.

Test: Restart and select Level 4. Every tab should show your content. Read your debug puzzle as a stranger would: is there really only one right choice?

Feature 3: Check Practice Answers

File: `challenges.py`, function `check_exercise`

Goal: A correct answer is accepted even with odd spacing, and a wrong answer gets feedback that points at the likely mistake.

Your Task: `check_exercise(level_id, user_answer)` returns `(is_correct, feedback)`. Today it only accepts an answer that matches an entry in `accepted_answers` *exactly*. Improve it so that:

1. spaces do not matter (`print( "Hi" )` counts as `print("Hi")`)

2. a wrong answer gets a specific message instead of "Not quite": for example a missing quote, a missing `)`, or a missing `:` at the end of the line

Hint: `.replace(" ", "")` removes spaces from a string. The `in` operator checks for a character, for example `")" in text`.

Test: In Level 1 Practice, type `print( "Hello, World!" )`. You should see a □. Then type `print(Hello, World!)` and expect a message about quotes. In Level 4, type `for i in range(5)` (no colon) and check what message you get.

Feature 4: Score the Quiz

File: `challenges.py`, function `score_quiz`

Goal: Count the right answers and tell the student which questions they missed.

Your Task: `score_quiz(level_id, user_answers)` gets the student's picks (in question order, `None` for a skipped question) and returns `(score, total, feedback)`. The feedback should say something like "You got 2 out of 3!" and name the question numbers that were missed. The student passes with 60% or more, which `app.py` already checks.

Hint: `range(len(questions))` gives you the positions 0, 1, 2, ..., so `questions[i]` and `user_answers[i]` are always the same question. People count from 1, lists count from 0.

Test: In Level 1, open the Quiz tab and answer everything correctly: you should see 3 out of 3 and a □ for quiz in the progress box. Then get one wrong and check that the feedback names it.

Feature 5: Unlock the Next Level

File: `progress.py`, function `unlock_next_level`

Goal: Finishing a level opens the next one.

Your Task: If the level is complete, add the next level in `LEVEL_ORDER` to `progress["unlocked"]` and return `progress`. (`is_level_complete` is ready to use.) The app calls your function after *every* correct activity, so it must not add a level twice, and it must not crash on the last level, which has no next one.

Hint: `LEVEL_ORDER.index("print")` gives a level's position in the list. A list has `append`, and `in` tells you whether something is already in it.

Test: Set `UNLOCK_ALL_FOR_TESTING` back to `False`, restart, and finish Level 1's practice, debug and quiz. The □ next to Level 2 should become a normal row you can open. Then play through to Level 5 to make sure nothing breaks.

Optional Extensions

Finished early? Try one of these:

- **Smarter tutor.** In `ai_helper.py`, rewrite `build_hint_prompt` to see how the tutor changes. Try to trick it into giving the answer, then tighten the rules.
- **Level 6.** Add a Lists level: a new id in `LEVEL_ORDER` and a new entry in `LEVELS`.
- **Remember progress.** Save the progress dictionary to a JSON file so a refresh does not reset it.
- **Lock icons.** Make `get_level_choices` show □ on levels the student has not unlocked.

- **AI-made puzzles.** Ask the AI for a new debug puzzle as JSON, then use *your own code* to check it is valid before the game uses it.